Lista dwukierunkowa - P1

Wygenerowano za pomocą Doxygen 1.18.0

1 Indeks klas 1

1.1 Lista klas . . . 1

2 Indeks plików 3

2.1 Lista plików . . . 3

3 Dokumentacja klas 5

3.1 Dokumentacja klasy DoublyLinkedList . . . 5

3.1.1 Opis szczegółowy . . . 6

3.1.2 Dokumentacja konstruktora i destruktora . . . 6

3.1.2.1 DoublyLinkedList() . . . 6

3.1.2.2 ~DoublyLinkedList() . . . 6

3.1.3 Dokumentacja funkcji składowych . . . 6

3.1.3.1 clear() . . . 6

3.1.3.2 displayBackward() . . . 6

3.1.3.3 displayForward() . . . 6

3.1.3.4 displayNext() . . . 6

3.1.3.5 displayPrev() . . . 7

3.1.3.6 insertAt() . . . 7

3.1.3.7 popBack() . . . 7

3.1.3.8 popFront() . . . 7

3.1.3.9 pushBack() . . . 7

3.1.3.10 pushFront() . . . 8

3.1.3.11 removeAt() . . . 8

3.2 Dokumentacja klasy Iterator . . . 8

3.2.1 Opis szczegółowy . . . 9

3.2.2 Dokumentacja konstruktora i destruktora . . . 9

3.2.2.1 Iterator() . . . 9

3.2.3 Dokumentacja funkcji składowych . . . 9

3.2.3.1 getData() . . . 9

3.2.3.2 isValid() . . . 9

3.2.3.3 next() . . . 9

3.2.3.4 prev() . . . 10

3.2.3.5 setNode() . . . 10

3.3 Dokumentacja klasy Node . . . 10

3.3.1 Opis szczegółowy . . . 10

3.3.2 Dokumentacja konstruktora i destruktora . . . 11

3.3.2.1 Node() . . . 11

3.3.3 Dokumentacja atrybutów składowych . . . 11

3.3.3.1 data . . . 11

3.3.3.2 next . . . 11

3.3.3.3 prev . . . 11

3.4 Dokumentacja klasy NodeFactory . . . 11

3.4.1 Opis szczegółowy . . . 12
3.4.2 Dokumentacja funkcji składowych . . . 12
3.4.2.1 createNode() . . . 12

4 Dokumentacja plików **13**

4.1 Dokumentacja pliku DoublyLinkedList.cpp . . . 13
4.1.1 Opis szczegółowy . . . 13
4.2 Dokumentacja pliku DoublyLinkedList.h . . . 13
4.2.1 Opis szczegółowy . . . 13
4.3 DoublyLinkedList.h . . . 14
4.4 Dokumentacja pliku Iterator.cpp . . . 14
4.4.1 Opis szczegółowy . . . 14
4.5 Dokumentacja pliku Iterator.h . . . 14
4.5.1 Opis szczegółowy . . . 15
4.6 Iterator.h . . . 15
4.7 Dokumentacja pliku main.cpp . . . 15
4.7.1 Opis szczegółowy . . . 15
4.7.2 Dokumentacja funkcji . . . 16
4.7.2.1 main() . . . 16
4.8 Dokumentacja pliku Node.cpp . . . 16
4.8.1 Opis szczegółowy . . . 16
4.9 Dokumentacja pliku Node.h . . . 16
4.9.1 Opis szczegółowy . . . 16
4.10 Node.h . . . 16
4.11 Dokumentacja pliku NodeFactory.cpp . . . 17
4.11.1 Opis szczegółowy . . . 17
4.12 Dokumentacja pliku NodeFactory.h . . . 17
4.12.1 Opis szczegółowy . . . 17
4.13 NodeFactory.h . . . 17

Skorowidz **19**

Rozdział 1

Indeks klas

1.1 Lista klas

Tutaj znajdują się klasy, struktury, unie i interfejsy wraz z ich krótkimi opisami:

DoublyLinkedList
: Klasa implementujaca liste dwukierunkowa dzialajaca na stercie 5

Iterator
: Klasa umozliwiajaca przechodzenie po elementach listy dwukierunkowej 8

Node
: Klasa reprezentujaca element (wezel) listy dwukierunkowej 10

NodeFactory
: Klasa fabryki odpowiedzialna za tworzenie nowych wezlow na stercie 11

Rozdział 2

Indeks plików

2.1 Lista plików

Tutaj znajduje się lista wszystkich plików wraz z ich krótkimi opisami:

DoublyLinkedList.cpp
: Implementacja metod klasy DoublyLinkedList 13

DoublyLinkedList.h
: Plik naglowkowy klasy DoublyLinkedList reprezentujacej liste dwukierunkowa 13

Iterator.cpp
: Implementacja metod klasy Iterator 14

Iterator.h
: Plik naglowkowy klasy Iterator realizujacej wzorzec projektowy Iteratora 14

main.cpp
: Glowny plik programu testujacy dzialanie listy dwukierunkowej 15

Node.cpp
: Implementacja metod klasy Node 16

Node.h
: Plik naglowkowy klasy Node reprezentujacej pojedynczy wezel listy dwukierunkowej 16

NodeFactory.cpp
: Implementacja metod klasy NodeFactory 17

NodeFactory.h
: Plik naglowkowy klasy NodeFactory realizujacej wzorzec projektowy Fabryki 17

Rozdział 3

Dokumentacja klas

3.1 Dokumentacja klasy DoublyLinkedList

Klasa implementujaca liste dwukierunkowa dzialajaca na stercie.

`#include <DoublyLinkedList.h>`

Metody publiczne

- DoublyLinkedList ()

  Konstruktor domyslny tworzacy pusta liste.
- ~DoublyLinkedList ()

  Destruktor zwalniajacy cala pamiec zajmowana przez liste na stercie.
- void pushFront (int val)

  Dodaje nowy element na poczatek listy.
- void pushBack (int val)

  Dodaje nowy element na koniec listy.
- void insertAt (int index, int val)

  Dodaje nowy element pod wskazany indeks.
- void popFront ()

  Usuwa element z poczatku listy.
- void popBack ()

  Usuwa element z konca listy.
- void removeAt (int index)

  Usuwa element spod wskazanego indeksu.
- void displayForward () const

  Wyswietla cala liste od poczatku do konca.
- void displayBackward () const

  Wyswietla liste w odwrotnej kolejnosci (od konca do poczatku).
- void displayNext ()

  Przesuwa wskaznik nawigacji i wyswietla nastepny element.
- void displayPrev ()

  Przesuwa wskaznik nawigacji i wyswietla poprzedni element.
- void clear ()

  Czysci cala liste i zwalnia pamiec na stercie.

3.1.1 Opis szczegółowy

Klasa implementujaca liste dwukierunkowa dzialajaca na stercie.

3.1.2 Dokumentacja konstruktora i destruktora

3.1.2.1 DoublyLinkedList()

```
DoublyLinkedList::DoublyLinkedList ()
```

Konstruktor domyslny tworzacy pusta liste.

3.1.2.2 ~DoublyLinkedList()

```
DoublyLinkedList::~DoublyLinkedList ()
```

Destruktor zwalniajacy cala pamiec zajmowana przez liste na stercie.

3.1.3 Dokumentacja funkcji składowych

3.1.3.1 clear()

```
void DoublyLinkedList::clear ()
```

Czysci cala liste i zwalnia pamiec na stercie.

3.1.3.2 displayBackward()

```
void DoublyLinkedList::displayBackward () const
```

Wyswietla liste w odwrotnej kolejnosci (od konca do poczatku).

3.1.3.3 displayForward()

```
void DoublyLinkedList::displayForward () const
```

Wyswietla cala liste od poczatku do konca.

3.1.3.4 displayNext()

```
void DoublyLinkedList::displayNext ()
```

Przesuwa wskaznik nawigacji i wyswietla nastepny element.

3.1.3.5 displayPrev()

```
void DoublyLinkedList::displayPrev ()
```

Przesuwa wskaznik nawigacji i wyswietla poprzedni element.

3.1.3.6 insertAt()

```
void DoublyLinkedList::insertAt (
            int index,
            int val)
```

Dodaje nowy element pod wskazany indeks.

Parametry

index	Pozycja, na ktora ma trafic element (liczac od 0).
val	Wartosc do wstawienia.

3.1.3.7 popBack()

```
void DoublyLinkedList::popBack ()
```

Usuwa element z konca listy.

3.1.3.8 popFront()

```
void DoublyLinkedList::popFront ()
```

Usuwa element z poczatku listy.

3.1.3.9 pushBack()

```
void DoublyLinkedList::pushBack (
            int val)
```

Dodaje nowy element na koniec listy.

Parametry

val	Wartosc do wstawienia.

3.1.3.10 pushFront()

```
void DoublyLinkedList::pushFront (
            int val)
```

Dodaje nowy element na poczatek listy.

Parametry

val	Wartosc do wstawienia.

3.1.3.11 removeAt()

```
void DoublyLinkedList::removeAt (
            int index)
```

Usuwa element spod wskazanego indeksu.

Parametry

index	Pozycja elementu do usuniecia (liczac od 0).

Dokumentacja dla tej klasy została wygenerowana z plików:

- DoublyLinkedList.h
- DoublyLinkedList.cpp

3.2 Dokumentacja klasy Iterator

Klasa umozliwiajaca przechodzenie po elementach listy dwukierunkowej.

```
#include <Iterator.h>
```

Metody publiczne

- Iterator (Node *startNode=nullptr)

  Konstruktor ustawiajacy iterator na wybrany wezel poczatkowy.
- void next ()

  Przesuwa iterator na nastepny wezel w liscie.
- void prev ()

  Przesuwa iterator na poprzedni wezel w liscie.
- bool isValid () const

  Sprawdza, czy iterator wskazuje na istniejacy wezel (czy nie wyszedl poza liste).
- int getData () const

  Zwraca wartosc przechowywana w aktualnym wezle.
- void setNode (Node *node)

  Ustawia iterator na nowy wybrany wezel.

3.2.1 Opis szczegółowy

Klasa umozliwiajaca przechodzenie po elementach listy dwukierunkowej.

3.2.2 Dokumentacja konstruktora i destruktora

3.2.2.1 Iterator()

```
Iterator::Iterator (
            Node * startNode = nullptr)
```

Konstruktor ustawiajacy iterator na wybrany wezel poczatkowy.

Parametry

startNode	Wskaznik na wezel, od ktorego rozpoczyna sie iteracja.

3.2.3 Dokumentacja funkcji składowych

3.2.3.1 getData()

```
int Iterator::getData () const
```

Zwraca wartosc przechowywana w aktualnym wezle.

Zwraca

Wartosc calkowita z aktualnego wezla.

3.2.3.2 isValid()

```
bool Iterator::isValid () const
```

Sprawdza, czy iterator wskazuje na istniejacy wezel (czy nie wyszedl poza liste).

Zwraca

true jesli wezel istnieje, false jesli wskaznik to nullptr.

3.2.3.3 next()

```
void Iterator::next ()
```

Przesuwa iterator na nastepny wezel w liscie.

3.2.3.4 prev()

```
void Iterator::prev ()
```

Przesuwa iterator na poprzedni wezel w liscie.

3.2.3.5 setNode()

```
void Iterator::setNode (
            Node * node)
```

Ustawia iterator na nowy wybrany wezel.

Parametry

node	Wskaznik na wezel, na ktory ma wskazywac iterator.

Dokumentacja dla tej klasy została wygenerowana z plików:

- Iterator.h
- Iterator.cpp

3.3 Dokumentacja klasy Node

Klasa reprezentujaca element (wezel) listy dwukierunkowej.

```
#include <Node.h>
```

Metody publiczne

- Node (int val)

  Konstruktor inicjalizujacy wezel podana wartoscia.

Atrybuty publiczne

- int data

  Wartosc przechowywana w wezle.
- Node * next

  Wskaznik na nastepny wezel w liscie.
- Node * prev

  Wskaznik na poprzedni wezel w liscie.

3.3.1 Opis szczegółowy

Klasa reprezentujaca element (wezel) listy dwukierunkowej.

3.3.2 Dokumentacja konstruktora i destruktora

3.3.2.1 Node()

```
Node::Node (
            int val)
```

Konstruktor inicjalizujacy wezel podana wartoscia.

Parametry

val	Wartosc calkowita do zapisania w wezle.

3.3.3 Dokumentacja atrybutów składowych

3.3.3.1 data

```
int Node::data
```

Wartosc przechowywana w wezle.

3.3.3.2 next

```
Node* Node::next
```

Wskaznik na nastepny wezel w liscie.

3.3.3.3 prev

```
Node* Node::prev
```

Wskaznik na poprzedni wezel w liscie.

Dokumentacja dla tej klasy została wygenerowana z plików:

- Node.h
- Node.cpp

3.4 Dokumentacja klasy NodeFactory

Klasa fabryki odpowiedzialna za tworzenie nowych wezlow na stercie.

```
#include <NodeFactory.h>
```

Statyczne metody publiczne

- static Node * createNode (int val)

  Tworzy nowy obiekt klasy Node na stercie.

3.4.1 Opis szczegółowy

Klasa fabryki odpowiedzialna za tworzenie nowych wezlow na stercie.

3.4.2 Dokumentacja funkcji składowych

3.4.2.1 createNode()

```
Node * NodeFactory::createNode (
            int val)  [static]
```

Tworzy nowy obiekt klasy Node na stercie.

Parametry

val	Wartosc, ktora zostanie przypisana do nowego wezla.

Zwraca

Wskaznik na nowo utworzony wezel na stercie.

Dokumentacja dla tej klasy została wygenerowana z plików:

- NodeFactory.h
- NodeFactory.cpp

Rozdział 4

Dokumentacja plików

4.1 Dokumentacja pliku DoublyLinkedList.cpp

Implementacja metod klasy DoublyLinkedList.

```
#include "DoublyLinkedList.h"
#include <iostream>
```

4.1.1 Opis szczegółowy

Implementacja metod klasy DoublyLinkedList.

4.2 Dokumentacja pliku DoublyLinkedList.h

Plik naglowkowy klasy DoublyLinkedList reprezentujacej liste dwukierunkowa.

```
#include "Node.h"
#include "NodeFactory.h"
#include "Iterator.h"
```

Komponenty

- class DoublyLinkedList

  Klasa implementujaca liste dwukierunkowa dzialajaca na stercie.

4.2.1 Opis szczegółowy

Plik naglowkowy klasy DoublyLinkedList reprezentujacej liste dwukierunkowa.

4.3 DoublyLinkedList.h

Idź do dokumentacji tego pliku.

```
00001 #pragma once
00002
00007
00008 #include "Node.h"
00009 #include "NodeFactory.h"
00010 #include "Iterator.h"
00011
00016 class DoublyLinkedList {
00017 private:
00018     Node* head;
00019     Node* tail;
00020     Node* currentNode;
00021     int size;
00022
00023 public:
00027     DoublyLinkedList();
00028
00032     ~DoublyLinkedList();
00033
00038     void pushFront(int val);
00039
00044     void pushBack(int val);
00045
00051     void insertAt(int index, int val);
00052
00056     void popFront();
00057
00061     void popBack();
00062
00067     void removeAt(int index);
00068
00072     void displayForward() const;
00073
00077     void displayBackward() const;
00078
00082     void displayNext();
00083
00087     void displayPrev();
00088
00092     void clear();
00093 };
```

4.4 Dokumentacja pliku Iterator.cpp

Implementacja metod klasy Iterator.

```
#include "Iterator.h"
```

4.4.1 Opis szczegółowy

Implementacja metod klasy Iterator.

4.5 Dokumentacja pliku Iterator.h

Plik naglowkowy klasy Iterator realizujacej wzorzec projektowy Iteratora.

```
#include "Node.h"
```

Komponenty

- class Iterator

  Klasa umozliwiajaca przechodzenie po elementach listy dwukierunkowej.

4.5.1 Opis szczegółowy

Plik naglowkowy klasy Iterator realizujacej wzorzec projektowy Iteratora.

4.6 Iterator.h

Idź do dokumentacji tego pliku.

```
00001 #pragma once
00002
00007
00008 #include "Node.h"
00009
00014 class Iterator {
00015 private:
00016     Node* current;
00017
00018 public:
00023     Iterator(Node* startNode = nullptr);
00024
00028     void next();
00029
00033     void prev();
00034
00039     bool isValid() const;
00040
00045     int getData() const;
00046
00051     void setNode(Node* node);
00052 };
```

4.7 Dokumentacja pliku main.cpp

Glowny plik programu testujacy dzialanie listy dwukierunkowej.

```
#include <iostream>
#include "DoublyLinkedList.h"
```

Funkcje

- int main ()

  Glowna funkcja programu testujaca wszystkie metody klasy DoublyLinkedList.

4.7.1 Opis szczegółowy

Glowny plik programu testujacy dzialanie listy dwukierunkowej.

4.7.2 Dokumentacja funkcji

4.7.2.1 main()

```
int main ()
```

Glowna funkcja programu testujaca wszystkie metody klasy DoublyLinkedList.

Zwraca

Kod powrotu programu (0 oznacza poprawne zakonczenie).

4.8 Dokumentacja pliku Node.cpp

Implementacja metod klasy Node.

```
#include "Node.h"
```

4.8.1 Opis szczegółowy

Implementacja metod klasy Node.

4.9 Dokumentacja pliku Node.h

Plik naglowkowy klasy Node reprezentujacej pojedynczy wezel listy dwukierunkowej.

Komponenty

- class Node

  Klasa reprezentujaca element (wezel) listy dwukierunkowej.

4.9.1 Opis szczegółowy

Plik naglowkowy klasy Node reprezentujacej pojedynczy wezel listy dwukierunkowej.

4.10 Node.h

Idź do dokumentacji tego pliku.

```
00001 #pragma once
00002
00007
00012
00013 class Node {
00014 public:
00015     int data;
00016     Node* next;
00017     Node* prev;
00018
00023     Node(int val);
00024 };
```

4.11 Dokumentacja pliku NodeFactory.cpp

Implementacja metod klasy NodeFactory.

```
#include "NodeFactory.h"
```

4.11.1 Opis szczegółowy

Implementacja metod klasy NodeFactory.

4.12 Dokumentacja pliku NodeFactory.h

Plik naglowkowy klasy NodeFactory realizujacej wzorzec projektowy Fabryki.

```
#include "Node.h"
```

Komponenty

- class NodeFactory

  Klasa fabryki odpowiedzialna za tworzenie nowych wezlow na stercie.

4.12.1 Opis szczegółowy

Plik naglowkowy klasy NodeFactory realizujacej wzorzec projektowy Fabryki.

4.13 NodeFactory.h

Idź do dokumentacji tego pliku.

```
00001 #pragma once
00002
00007
00008 #include "Node.h"
00009
00014
00015 class NodeFactory {
00016 public:
00022     static Node* createNode(int val);
00023 };
```

Skorowidz

~DoublyLinkedList
    DoublyLinkedList, 6

clear
    DoublyLinkedList, 6
createNode
    NodeFactory, 12

data
    Node, 11
displayBackward
    DoublyLinkedList, 6
displayForward
    DoublyLinkedList, 6
displayNext
    DoublyLinkedList, 6
displayPrev
    DoublyLinkedList, 6
DoublyLinkedList, 5
    ~DoublyLinkedList, 6
    clear, 6
    displayBackward, 6
    displayForward, 6
    displayNext, 6
    displayPrev, 6
    DoublyLinkedList, 6
    insertAt, 7
    popBack, 7
    popFront, 7
    pushBack, 7
    pushFront, 7
    removeAt, 8
DoublyLinkedList.cpp, 13
DoublyLinkedList.h, 13

getData
    Iterator, 9

insertAt
    DoublyLinkedList, 7
isValid
    Iterator, 9
Iterator, 8
    getData, 9
    isValid, 9
    Iterator, 9
    next, 9
    prev, 9
    setNode, 10
Iterator.cpp, 14
Iterator.h, 14

main
    main.cpp, 16
main.cpp, 15
    main, 16

next
    Iterator, 9
    Node, 11
Node, 10
    data, 11
    next, 11
    Node, 11
    prev, 11
Node.cpp, 16
Node.h, 16
NodeFactory, 11
    createNode, 12
NodeFactory.cpp, 17
NodeFactory.h, 17

popBack
    DoublyLinkedList, 7
popFront
    DoublyLinkedList, 7
prev
    Iterator, 9
    Node, 11
pushBack
    DoublyLinkedList, 7
pushFront
    DoublyLinkedList, 7

removeAt
    DoublyLinkedList, 8

setNode
    Iterator, 10